

1.1 有关算法的基本概念

Algorithm + Data Structure = **Programming**

- 好的算法
 - (1) 提高求解问题的效率
 - (2) 节省存储空间
- 算法的研究目标
 - (1) 实际问题→建模并寻找算法
 - (2) 设计的算法→算法的评价
 - (3) 算法类（排序）→问题复杂度估计
 - (4) 问题类(NP难)→能够求解的边界

算法设计技术

算法分析方法

问题复杂度分析

计算复杂性理论



1.1 有关算法的基本概念

■ 算法及时间复杂度估算

➤ 算法中涉及的基本概念

(1) 问题：需要回答的一般性提问，通常含若干参数。

(2) 问题描述

- 定义问题参数(集合,变量,函数,序列等)
- 说明每个参数的取值范围及参数间的关系
- 定义问题的解
- 说明解满足的条件(优化目标或约束条件)

(3) 问题实例：参数的一组赋值可得到问题的一个实例。

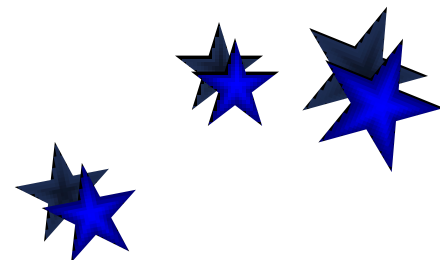


1.1 有关算法的基本概念

(4) 算法：有限条指令的序列，这个指令序列确定了解决某个问题的一系列运算或操作。

(5) 算法 A 解决问题 P

- 把问题 P 的**任何实例**作为算法 A 的输入
- 每步计算是确定性的
- A 能够在有限步停机
- 输出该实例的正确解



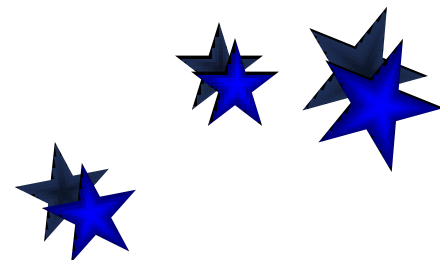
1.1 有关算法的基本概念

➤ 算法时间复杂度估算

(1) 算法时间复杂度: 针对指定基本运算, 计数算法所做该运算的次数, 通常是问题输入规模的函数。

(2) 输入规模: 输入串编码长度。例如:

- 排序: 数组中元素个数 n
- 检索: 被检索数组的元素个数 n
- 整数乘法: 两个整数的位数 m, n
- 矩阵相乘: 矩阵的行列数 i, j, k
- 图的遍历: 图的顶点数 n , 边数 m



1.1 有关算法的基本概念

(3) 基本运算：比较、加法、乘法、置指针, 交换, 赋值...
例如：

- 排序: 元素之间的比较
- 检索: 被检索元素 x 与数组元素的比较
- 整数乘法: 每位数字相乘(位乘) 1 次 m 位和 n 位整数相乘要做 mn 次位乘
- 矩阵相乘: 每对元素乘 1 次 $i \times j$ 矩阵与 $j \times k$ 矩阵相乘要做 ijk 次乘法
- 图的遍历: 置指针

(4) 算法基本运算次数可表为输入规模的函数。

(5) 给定问题和基本运算就决定了一个算法类。



1.1 有关算法的基本概念

(6) 算法时间复杂度估算方法

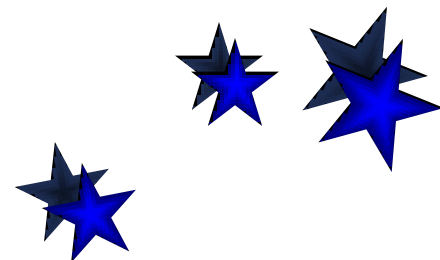
- 方法1：相同输入规模的不同实例，算法的基本运算次数也不一样，可分析如下三种时间复杂度：

*最好时间复杂度

*最坏时间复杂度

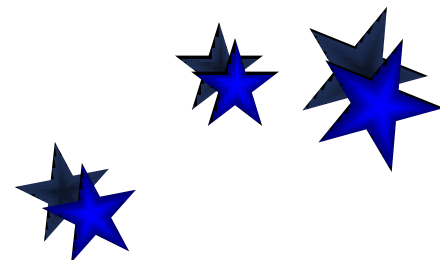
*平均时间复杂度

- 方法2：渐进时间复杂度（1.3节）



1.1 有关算法的基本概念

- 设一个算法的输入规模为 n ， D_n 是所有输入的集合，任一输入 $I \in D_n$ ， $P(I)$ 是 I 出现的概率，有 $\sum P(I) = 1$ ， $T(I)$ 是算法在输入 I 下所执行的基本语句次数，则：
 - (1) 算法的最好情况为： $G(n) = \min_{I \in D_n} \{T(I)\}$ ，是指算法在所有输入 I 下所执行基本语句的最少次数。
 - (2) 算法的最坏情况为： $W(n) = \max_{I \in D_n} \{T(I)\}$ ，是指算法在所有输入 I 下所执行基本语句的最大次数。

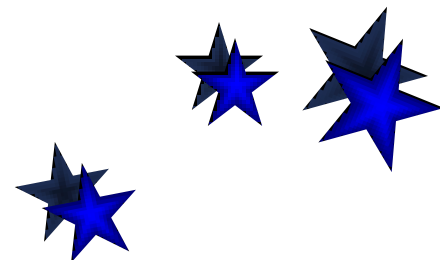


1.1 有关算法的基本概念

(3) 该算法的平均执行时间为

$$A(n) = \sum_{I \in D_n} P(I) * T(I)$$

- * 也就是说算法的平均情况是指用各种可能输入下的基本语句执行次数的带权平均值。
- * 必须知道所有输入实例出现的概率，即所有输入的分布情况。通常假定输入是平均分布的（等概率）。



1.1 有关算法的基本概念

- 最好、最坏、平均时间复杂度举例

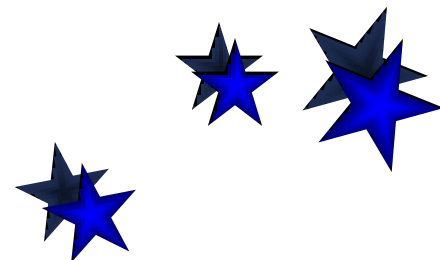
例题8：检索问题——线性检索法

输入：非降顺序排列的数组 L , 元素个数 n , 待检索数 x

输出：位置 i ，若 x 在 L 中, i 是 x 首次出现的下标；

否则 $i = 0$

基本运算： x 与 L 中元素的比较运算



1.1 有关算法的基本概念

- 实现算法1

$i=1$, 将 x 与 $L[i]$ 比较. 如果 $x=L[i]$, 则算法停止, 输出 i ; 如果不等, 则把 i 加1, 继续 x 与 $L[i]$ 的比较, 如果 $i>n$, 则停机并输出0.

实例

1

2

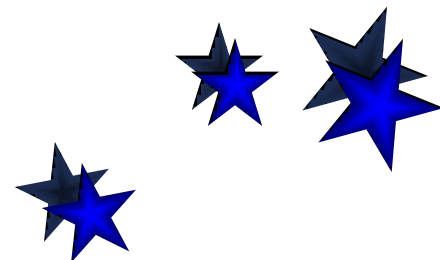
3

4

5

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 5 次



1.1 有关算法的基本概念

```
int Find1(double L[],int n,double x)
{ int i=1;
  while (i<=n)
  { if (L[i]==x) break;
    i++;
  }
  if (i<=n) return i;
  else return 0;
}
```



1.1 有关算法的基本概念

(1) 分析该算法在等概率情况下**成功查找**到值为 x 的元素的

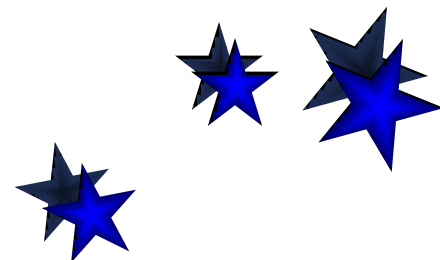
✓ 最好时间复杂度

第一个元素 $L[1]$ 等于 x ，此时基本语句仅执行一次，此时呈现**最好的情况**，即 $G(n)=O(1)$

✓ 最坏时间复杂度

当 L 中最后一个元素 $L[n-1]$ 等于 x ，此时基本语句执行 n 次，此时呈现**最坏的情况**，即 $W(n)=O(n)$ 。

✓ 平均时间复杂度



1.1 有关算法的基本概念

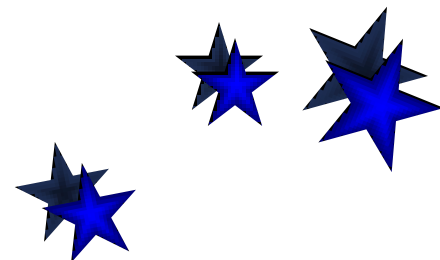
✓ 平均时间复杂度

实例

1	2	3	4	5
---	---	---	---	---

对于其他情况，假设查找每个元素的概率相同，则 $P(L[i])=1/n$
($1 \leq i \leq n$)，而成功找到 $L[i]$ 元素时基本语句正好执行 i 次，所以
:

$$A(n) = \sum_{i=1}^n \frac{1}{n} (i+1) = \frac{1}{n} \sum_{i=1}^n (i+1) = \frac{n+1}{2} = O(n)。$$



1.1 有关算法的基本概念

(2) 该问题可设计多少个输入实例进行复杂度分析？

实例

1

2

3

4

5

查找成功： n 个实例，比较次数为 i

$x = L[1], x = L[2], \dots, x = L[n]$

查找不成功： $n+1$ 个实例，比较次数均为 n

$x < L[1], L[1] < x < L[2], L[2] < x < L[3], \dots, L[n] < x$

最坏情况下时间： $W(n) = n$

最坏的输入： x 不在 L 中或 $x = L[n]$ ，要做 n 次比较



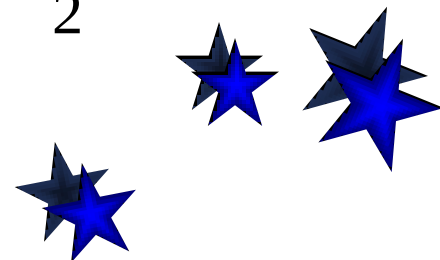
1.1 有关算法的基本概念

(3) 假设被查值 x 在数组 L 中的概率是 q ，求算法的时间复杂度。

- 对于成功查找，等概率情况下元素 $L[i]$ 被查找到的概率 $P(L[i])=q/n$ ，成功找到 $L[i]$ 元素时基本操作正好执行 i 次。
- 对于不成功查找，其概率为 $1-q$ ，不成功查找时基本操作正好执行 n 次。

$$\begin{aligned} A(n) &= \sum_{I \in D_n} P(I) * T(I) \\ &= \sum_{i=0}^n P(I) * T(I) = \sum_{i=0}^{n-1} \frac{q}{n} (i+1) + (1-q)n = \frac{(n+1)q}{2} + (1-q)n \end{aligned}$$

若 $q=1/2$ ，则 $A(n) = [(n+1)/4] + n/2 \approx 3n/4$ 。



1.1 有关算法的基本概念

- 实现算法2——利用**非降序排列**的特性

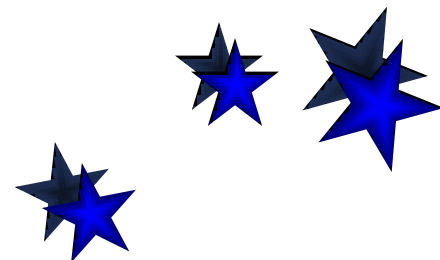
$j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输出 j ; 如果 $x > L[j]$, 则把 j 加1, 继续 x 与 $L[j]$ 的比较; 如果 $x < L[j]$, 则停机并输出0. 如果 $j > n$, 则停机并输出 0.

实例

1	2	3	4	5
---	---	---	---	---

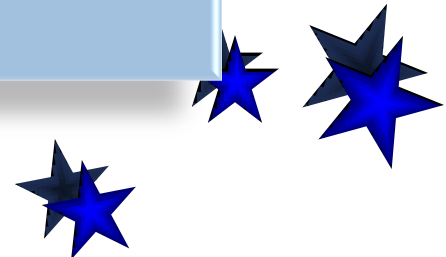
$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 3 次



1.1 有关算法的基本概念

```
int Find2(double L[],int n,double x)
{ int i=1;
  while (i<=n)
  { if (L[i]>=x) break;
    i++;
  }
  if (i<=n&&L[i]==x) return i;
  else return 0;
}
```



1.1 有关算法的基本概念

检索算法2的时间估计

最好情况下: $x = L[1], x < L[1]$

$$G(n) = 1$$

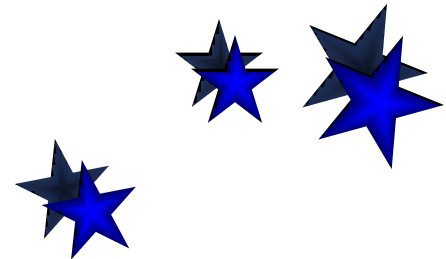
最坏情况下: $x = L[n], L[n-1] < x < L[n], L[n] < x$

$$W(n) = n$$

平均情况下: 输入实例的概率分布: 假设 x 在 L 中每个位置 (成功) 与空隙 (不成功) 的概率都相等



改进检索算法平均时间复杂度是多少?



1.1 有关算法的基本概念

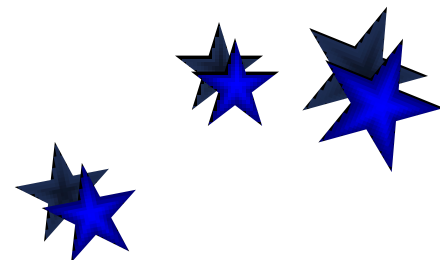
小结:

- 算法最好、最坏和平均情况下的时间复杂度定义
- 如何计算上述时间复杂度

练习:

- 请给出一个效率最高的百钱买百鸡问题的求解方案

问题叙述：“鸡翁一，值钱五；鸡母一，值钱三；鸡雏三，值钱一。百钱买百鸡，问鸡翁、母、雏各几何？”



1.2 算法的伪码描述（自学）

- 伪码不是程序代码，只是用于描述算法的主要步骤。
- 伪码中的关键字

赋值语句：←

分支语句：if ...then ... [else...]

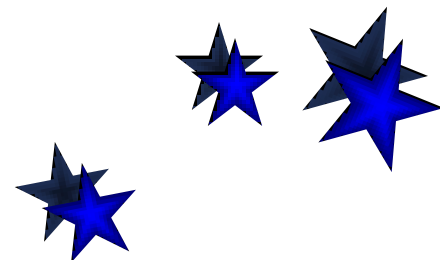
循环语句：while, for, repeat until

转向语句：goto

输出语句：return

调用：直接写过程的名字

注释：//...



1.2 算法的伪码描述（自学）

- 使用伪码描述算法举例

例1：求最大公约数—辗转相除法

算法1.1 Euclid(m, n)

输入：非负整数 m, n ，其中 m 与 n 不全为0

输出： m 与 n 的最大公约数

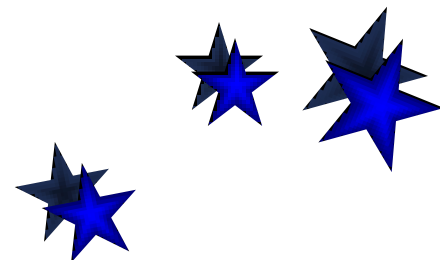
1. while $m > 0$ do

2. $r \leftarrow n \bmod m$

3. $n \leftarrow m$

4. $m \leftarrow r$

5. return n



1.2 算法的伪码描述（自学）

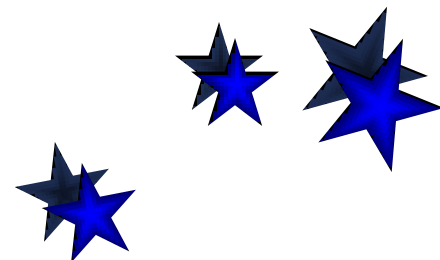
例2：改进的顺序检索

算法1.2 Search(L, x)

输入：数组 $L[1..n]$ ，其元素按照从小到大排列，数 x 。

输出：若 x 在 L 中，输出 x 的位置下标 j ；否则输出0。

1. $j \leftarrow 1$
2. while $j \leq n$ and $x > L[j]$ do $j \leftarrow j + 1$
3. if $x < L[j]$ or $j > n$ then $j \leftarrow 0$
4. return j



1.2 算法的伪码描述（自学）

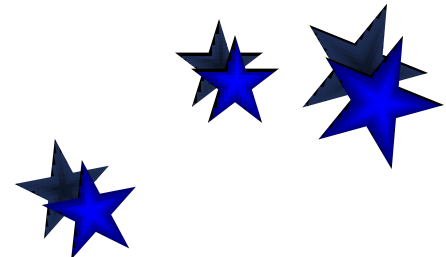
例3：插入排序

算法1.3 Insert Sort (A, n)

输入： n 个数的数组 A

输出：按照递增顺序排好序的数组 A

1. for $j \leftarrow 2$ to n do
2. $x \leftarrow A[j]$
3. $i \leftarrow j-1$ //3-7 行把 $A[j]$ 插入 $A[1..j-1]$
4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$



1.2 算法的伪码描述（自学）

例4：二分归并排序

算法1.4 MergeSort (A, p, r)

输入：数组 $A[p..r]$

输出：按递增顺序排序的数组 A

- 1. if $p < r$**
- 2. then $q \leftarrow \text{int}((p+r)/2)$**
- 3. MergeSort (A, p, q)**
- 4. MergeSort ($A, q+1, r$)**
- 5. Merge (A, p, q, r)**

MergeSort有递归调用，也调用Merge过程，Merge过程另行定义。

1.2 算法的伪码描述（自学）

例5：分析算法1.5 的功能

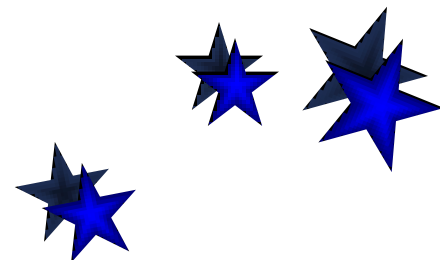
算法1.5：A

输入：实数的数组 $P[0..n]$ ，实数 x

输出： y

```
1.  $y \leftarrow P[0]$ ;  $power \leftarrow 1$   
2. for  $i \leftarrow 1$  to  $n$  do  
3.    $power \leftarrow power * x$   
4.    $y \leftarrow y + P[i] * power$   
5. return  $y$ 
```

- 输入 $P[0..n]$ 是 n 次多项式 $P(x)$ 的系数
- 算法 A 计算该多项式在 x 的值



1.3 算法的数学基础

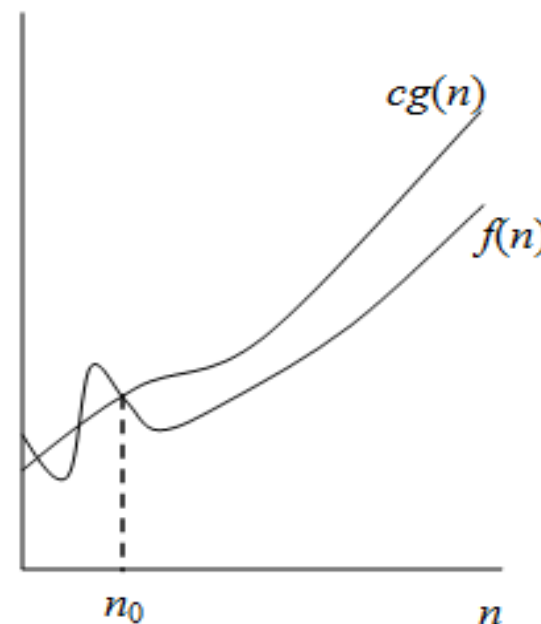
■ 函数的渐近的界

(1) 大O符号

• **定义**：设 f 和 g 是定义域为自然数集 N 上的函数。若存在正数 c 和 n_0 ，使得对一切 $n \geq n_0$ 有

$$0 \leq f(n) \leq c g(n)$$

成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$



(b) $f(n) = O(g(n))$



1.3 算法的数学基础

•大O符号举例

$f(n) = n^2 + n$, 则

* $g(n)=n^2$, $f(n)=O(g(n))$, 取 $c = 2$, $n_0=1$ 即可。

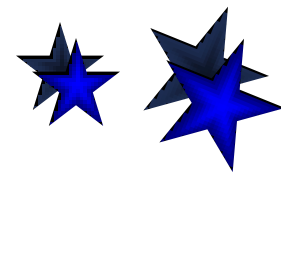
* $g(n)=n^3$, $f(n)=O(g(n))$, 取 $c = 1$, $n_0=2$ 即可。

➤ $f(n) = O(g(n))$, $f(n)$ 的阶不高于 $g(n)$ 的阶。

➤ 可能存在多个正数 c , 只要指出一个即可。

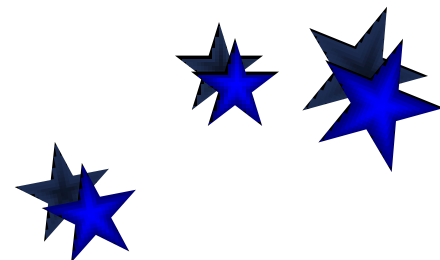
➤ 对 n_0 前面有限个 n 值可以不满足不等式。

➤ 常函数（循环次数已知）可以写作 $O(1)$ 。



1.3 算法的数学基础

- 大 O 符号用来描述增长率的上界，表示 $f(n)$ 的增长最多像 $g(n)$ 增长的那样快，也就是说，当输入规模为 n 时，算法消耗时间的最大值。
- 这个上界的阶越低结果就越有价值，所以，对于多项式 $10n^2+4n+2$ ， $O(n^2)$ 比 $O(n^4)$ 有价值。
- 一个算法的时间用大 O 符号表示时，总是采用最有价值的 $g(n)$ 表示，称之为“紧凑上界”或“紧确上界”。



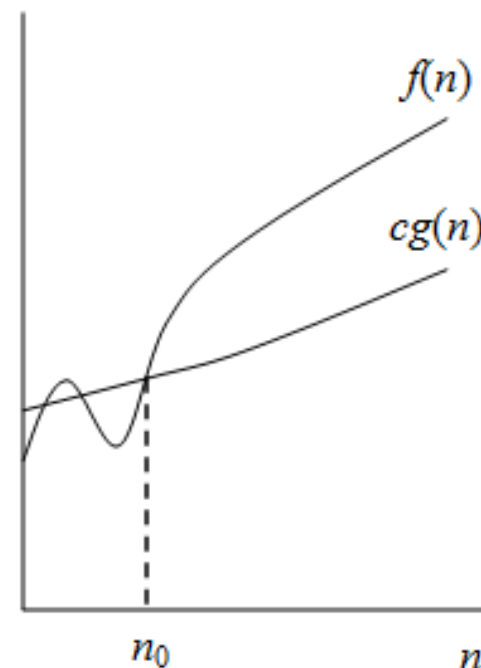
1.3 算法的数学基础

(2) 大 Ω 符号

• **定义**：设 f 和 g 是定义域为自然数集 N 上的函数。若存在正数 c 和 n_0 ，使得对一切 $n \geq n_0$ 有

$$0 \leq cg(n) \leq f(n)$$

成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$



(c) $f(n) = \Omega(g(n))$



1.3 算法的数学基础

• 大 Ω 符号举例

$f(n) = n^2 + n$, 则

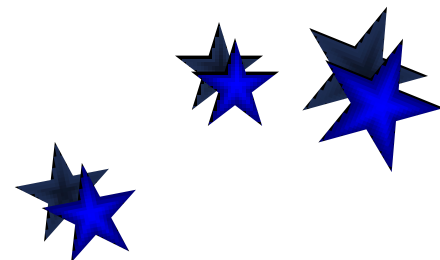
* $g(n)=n^2$, $f(n)=\Omega(g(n))$, 取 $c = 1$, $n_0=0$ 即可。

* $g(n)=100n$, $f(n)=\Omega(g(n))$, 取 $c = 1/100$, $n_0=1$ 即可

➤ $f(n)=\Omega(g(n))$, $f(n)$ 的阶不低于 $g(n)$ 的阶。

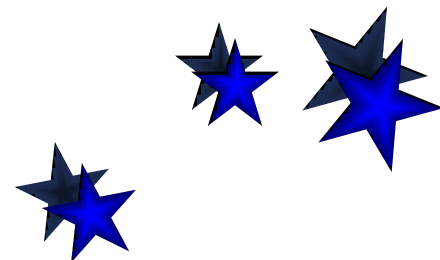
➤ 可能存在多个正数 c , 只要指出一个即可。

➤ 对 n_0 前面有限个 n 值可以不满足不等式。



1.3 算法的数学基础

- 大 Ω 符号用来描述增长率的下界，表示 $f(n)$ 的增长最少像 $g(n)$ 增长的那样快，也就是说，当输入规模为 n 时，算法消耗时间的最小值。
- 这个下界的阶越高结果就越有价值，所以，对于多项式 $10n^2+4n+2$ ， $\Omega(n^2)$ 比 $\Omega(n)$ 有价值。
- 一个算法的时间用大 Ω 符号表示时，总是采用最有价值的 $g(n)$ 表示，称之为“紧凑下界”或“紧确下界”。



1.3 算法的数学基础

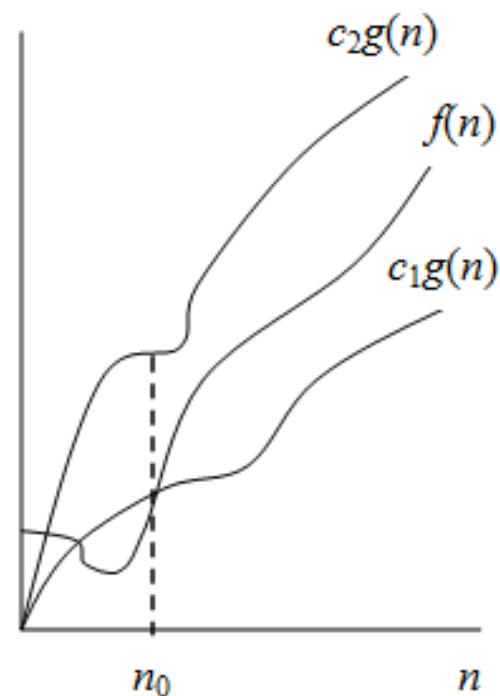
(3) Θ 符号

• **定义：** 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$ ，则记作 $f(n) = \Theta(g(n))$ 。

例子： $f(n) = n^2 + n$, $g(n) = n^2$ ，那么有 $f(n) = \Theta(g(n))$

1. $f(n)$ 的阶与 $g(n)$ 的阶相等.

2. 对 n_0 前面有限个 n 值可以不满足条件.



(a) $f(n) = \Theta(g(n))$



1.3 算法的数学基础

- Θ 符号举例：素数测试

算法 **PrimalityTest**(n)

输入： n , 大于2的奇整数

输出： **true** 或者 **false**

1. $s \leftarrow \text{int}(n^{1/2})$
2. **for** $j \leftarrow 2$ **to** s
3. **if** j 整除 n
4. **then return false**
5. **return true**

问题：

若 $n^{1/2}$ 可在 $O(1)$ 计算，
基本运算是整除，以下表示是否正确？

$$W(n) = O(n^{1/2})$$

$$W(n) = \Omega(n^{1/2})$$

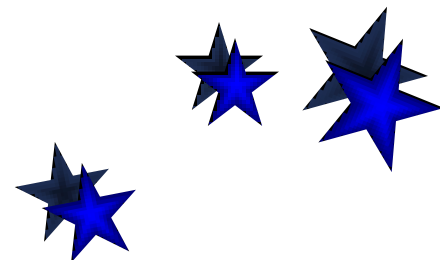
$$W(n) = \Theta(n^{1/2})$$

Θ 符号比大 O 符号和大 Ω 符号都精确， $f(n) = \Theta(g(n))$ ，当且仅当 $g(n)$ 既是 $f(n)$ 的上界又是 $f(n)$ 的下界。



1.3 算法的数学基础

- 大 Ω 符号用来描述增长率的下界，表示 $f(n)$ 的增长最少像 $g(n)$ 增长的那样快，也就是说，当输入规模为 n 时，算法消耗时间的最小值。
- 这个下界的阶越高结果就越有价值，所以，对于多项式 $10n^2+4n+2$ ， $\Omega(n^2)$ 比 $\Omega(n)$ 有价值。
- 一个算法的时间用大 Ω 符号表示时，总是采用最有价值的 $g(n)$ 表示，称之为“紧凑下界”或“紧确下界”。



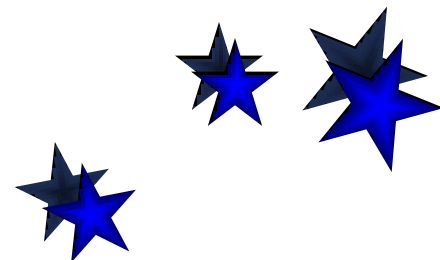
1.3 算法的数学基础

(4) 小o符号

- 定义：设 f 和 g 是定义域为自然数集 $\mathbf{N}$ 上的函数。若对于任意正数 c 都存在 n_0 ，使得对一切 $n \geq n_0$ 有

$$0 \leq f(n) < c g(n)$$

成立, 则记作: $f(n) = o(g(n))$



1.3 算法的数学基础

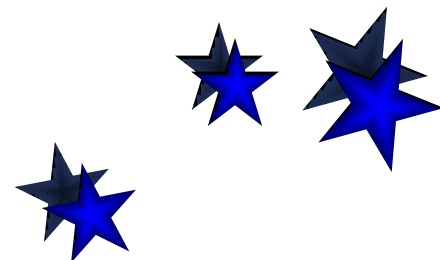
例子: $f(n)=n^2+n$, $g(n)=n^3$ 则 $f(n)=o(g(n))$

- $c \geq 1$ 显然成立, 因为 $n^2+n < cn^3$ ($n_0=2$)
- 任给 $1 > c > 0$, 取 $n_0 = \lceil 2/c \rceil$, (即 $n_0 > 2$)

当 $n \geq n_0$ 时, $cn \geq cn_0 > 2$

所以, $n^2+n < 2n^2 < cn \times n^2 = cn^3$

1. $f(n) = o(g(n))$, $f(n)$ 的阶低于 $g(n)$ 的阶。
2. 对不同正数 c , n_0 不一样。 c 越小 n_0 越大。
3. 对 n_0 前面有限个 n 值可以不满足不等式。



1.3 算法的数学基础

(5) 小 ω 符号

- 定义：设 f 和 g 是定义域为自然数集 $\mathbf{N}$ 上的函数。若对于任意正数 c 都存在 n_0 ，使得对一切 $n \geq n_0$ 有

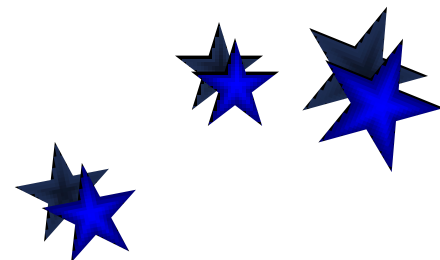
$$0 \leq cg(n) < f(n)$$

成立, 则记作: $f(n) = \omega(g(n))$

例子: $f(n)=n^2+n$, $g(n)=n$ 则 $f(n)=\omega(g(n))$

- 对于任意 $c > 0$, 取 $n_0 = c$, 当 $n \geq n_0$ 时

当 $n \geq n_0$ 时, $n^2+n > cn$, 即 $f(n)=\omega(g(n))$



1.3 算法的数学基础

- 注意：不能写 $f(n) = \omega(n^2)$ ，因为

(1) $c=1$ 时，取 $n_0=1$ ，当 $n \geq n_0$ 时， $n^2+n > cn^2=n^2$

(2) $c=2$ 时，不存在 n_0 ，使得当 $n \geq n_0$ 时， $n^2+n > cn^2=2n^2$

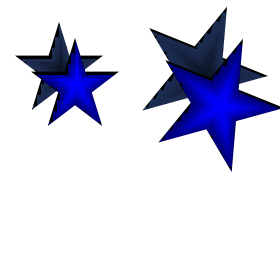
.....

不满足对任意 c 都存在 n_0 ，使得 $n \geq n_0$ ， $f(n) > cg(n)$ 成立

1. $f(n)=\omega(g(n))$ ， $f(n)$ 的阶 **高于** $g(n)$ 的阶。

2. 对不同的正数 c ， n_0 不等， c 越大 n_0 越大。

3. 对 n_0 前面有限个 n 值可以不满足不等式。

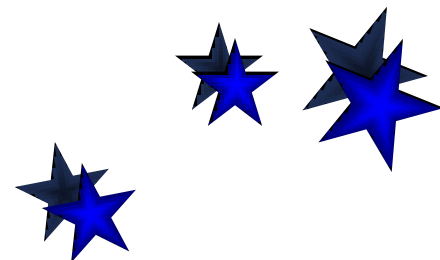


1.3 算法的数学基础

■ 有关函数渐近的界的定理

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

- (1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0$, 那么 $f(n) = O(g(n))$ 。
- (2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty$, 那么 $f(n) = \Omega(g(n))$ 。
- (3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, 常数 $c > 0$, 那么 $f(n) = \Theta(g(n))$ 。
- (4) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$ 。
- (5) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$ 。



1.3 算法的数学基础

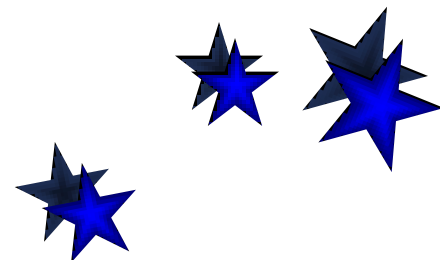
- 定理1.1的应用——估计函数的阶

例4 设 $f(n) = \frac{1}{2}n^2 - 3n$, 证明 $f(n) = \Theta(n^2)$.

证 因为

$$\lim_{n \rightarrow +\infty} \frac{f(n)}{n^2} = \lim_{n \rightarrow +\infty} \frac{\frac{1}{2}n^2 - 3n}{n^2} = \frac{1}{2}$$

根据定理1.1有 $f(n) = \Theta(n^2)$.



1.3 算法的数学基础

- 一些重要结论：可以证明

(1) 多项式函数的阶低于指数函数

$$n^d = o(r^n), \quad r > 1, \quad d > 0$$

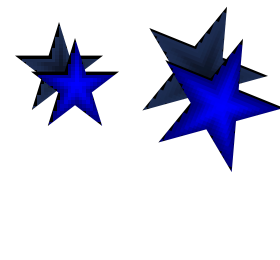
$$\lim_{n \rightarrow \infty} \frac{n^d}{r^n} = \lim_{n \rightarrow \infty} \frac{dn^{d-1}}{r^n \ln r} = \lim_{n \rightarrow \infty} \frac{d(d-1)n^{d-2}}{r^n (\ln r)^2} = \dots = \lim_{n \rightarrow \infty} \frac{d!}{r^n (\ln r)^d} = 0$$

分子分母
求导数

(2) 对数函数的阶低于幂函数的阶

$$\ln n = o(n^d), \quad d > 0$$

$$\lim_{n \rightarrow \infty} \frac{\ln n}{n^d} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{dn^{d-1}} = \lim_{n \rightarrow \infty} \frac{1}{dn^d} = 0$$



1.3 算法的数学基础

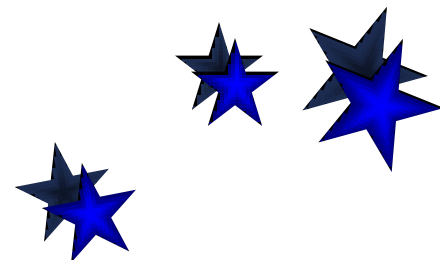
定理1.2 设 f, g, h 是定义域为自然数集合的函数,

(1) 如果 $f=O(g)$ 且 $g=O(h)$, 那么 $f=O(h)$.

(2) 如果 $f=\Omega(g)$ 且 $g=\Omega(h)$, 那么 $f=\Omega(h)$.

(3) 如果 $f=\Theta(g)$ 和 $g=\Theta(h)$, 那么 $f=\Theta(h)$.

定理1.2说明: 函数的阶之间的关系具有传递性



1.3 算法的数学基础

- 定理1.2的应用

例5：按照阶从高到低排序以下函数

$$f(n)=(n^2+n)/2, \quad g(n)=10n$$

$$h(n)=1.5^n, \quad t(n)=n^{1/2}$$

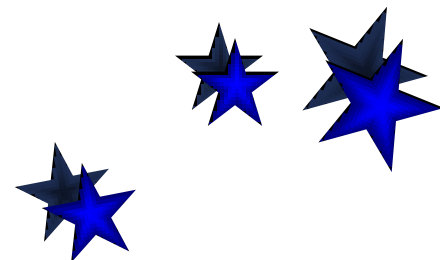
解：根据定理1.1，可给出如下函数关系

$$h(n) = \omega(f(n)),$$

$$f(n) = \omega(g(n)),$$

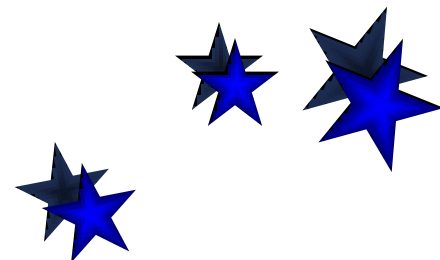
$$g(n) = \omega(t(n)),$$

排序结果： $h(n), f(n), g(n), t(n)$



1.3 算法的数学基础

- 定理1.3 假设函数 f 和 g 的定义域为自然数集, 若对某个其它函数 h , 有 $f=O(h)$ 和 $g=O(h)$, 那么 $f+g=O(h)$
- 定理1.3可以推广到有限个函数: 算法由有限步骤构成. 若每一步的时间复杂度函数的上界都是 $h(n)$, 那么该算法的时间复杂度函数可以写作 $O(h(n))$.
- 算法的时间复杂度是各步操作时间之和, 在常数步的情况下取最高阶的函数即可.



1.3 算法的数学基础

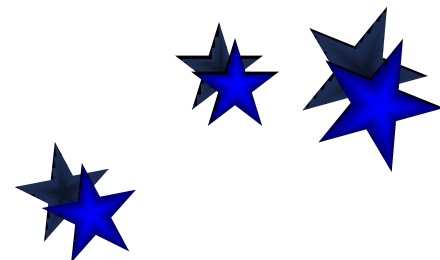
■ 几类重要的函数——自学

阶的高低

至少指数级: $2^n, 3^n, n!, \dots$

多项式级: $n, n^2, n \log n, n^{1/2}, \dots$

对数多项式级: $\log n, \log^2 n, \log \log n, \dots$



1.3 算法的数学基础

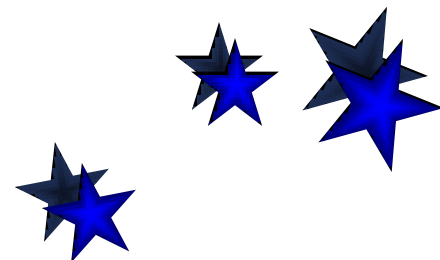
- 常见函数的阶从高到低的排序

$$2^{2^n}, n!, n2^n, (3/2)^n, (\log n)^{\log n} = n^{\log \log n},$$

$$n^3, \log(n!) = \Theta(n \log n), n = \Theta(2^{\log n}),$$

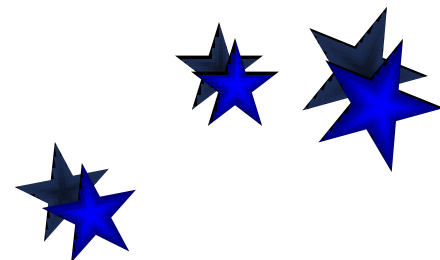
$$\log^2 n, \log n, \sqrt{\log n}, \log \log n,$$

$$n^{1/\log n} = \Theta(1)$$



1.3 算法的数学基础

- 小结
 - 几类常用函数的阶的性质
 - ✓ 对数函数
 - ✓ 指数函数
 - ✓ 阶乘函数
 - ✓ 取整函数
 - 如何利用上述性质估计函数的阶？



1.3 算法的数学基础

■ 算法复杂度分析方法

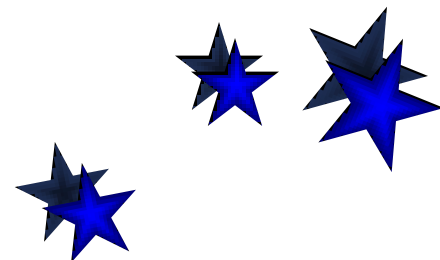
- 算法分类

- (1) 循环类（迭代）算法

- 该类算法的时间复杂度通常表现为算法中基本运算的执行次数的累加和。

- (2) 递归类算法

- 该类算法的时执行时间通常表示为递归方程形式，因此需要对递归方程进行求解，找出算法的时间复杂度表示。



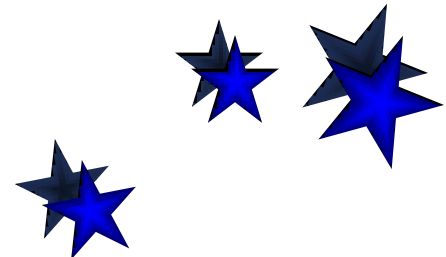
1.3 算法的数学基础

- 循环类算法时间复杂度分析
 - 循序次数直接依赖问题规模 n :

```
x=0;  
for(k=1;k<=n;k++)  
    x=x+1;  
for(i=1;i<=n;i++)  
    for(j=1;j<=n;j++)  
        y=y+1;
```

最内层语句执行次数: n^2

时间复杂度: $O(n^2)$



1.3 算法的数学基础

- 循环次数间接依赖问题规模 n

$x=1;$

$for(i=1;i\leq n;i++)$

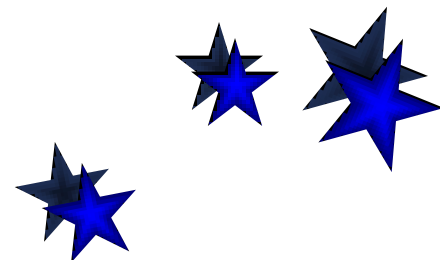
$for(j=1;j\leq i;j++)$

$for(k=1;k\leq j;k++)$

$x=x+1;$

$$\begin{aligned}\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 &= \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right) \\ &= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right)\end{aligned}$$

时间复杂度: $O(n^3)$



1.3 算法的数学基础

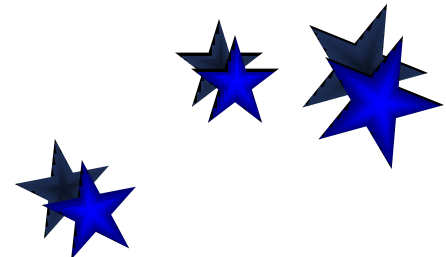
- 举例1：冒泡排序：数组中的n个元素由小到大进行排序

```
1. template <class T>
2. void bubble(Type A[],int n)
3. {
4.     int i,k;
5.     for (k=n-1;k>0;k--) {
6.         for (i=0;i<k;i++)
7.             if (A[i] > A[i+1]) {
8.                 swap(A[i],A[i+1]);
9.             }
10.        }
11.    }
12. }
```

$$1 + 2 + 3 + \dots + (n-1)$$

$$= \frac{n(n-1)}{2}$$

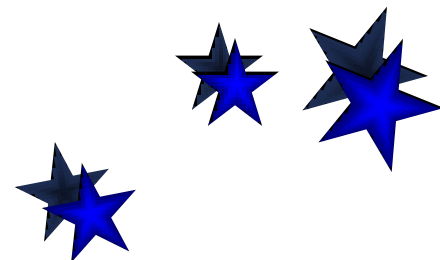
时间复杂度： $O(n^2)$



1.3 算法的数学基础

举例2：选手的竞技淘汰比赛——锦标赛排序

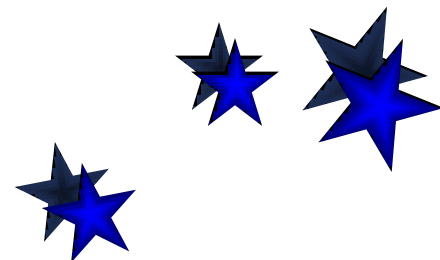
有 $n = 2^k$ 位选手进行竞技淘汰比赛，最后决出冠军的选手。假定函数：*int comp(Type mem1, Type mem2)*模拟两位选手的比赛，若*mem1*胜则返回1，否则返回0。并假定可以在常数时间*c*内完成函数*comp*的执行。



1.3 算法的数学基础

```
1. Type game(Type group[],int n)
2. {
3.   int j,i = n;
4.   while (i>1) { /*n=2^k 共进行k次循环*/
5.     i = i / 2;
6.     for (j=0;j<i;j++) /*每一轮需要进行i/2次比赛*/
7.       if (comp(group[j+i],group[j]))
8.         group[j] = group[j+i];
9.   }
10.  return group[0];
11. }
```

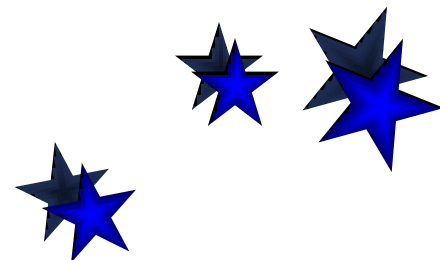
$$T(n) = \frac{n}{2} + \frac{n}{4} + \cdots + \frac{n}{2^k} = n \sum_{i=1}^k \frac{1}{2^i} \quad (k = \log n)$$



1.3 算法的数学基础

- 举例3：洗牌程序

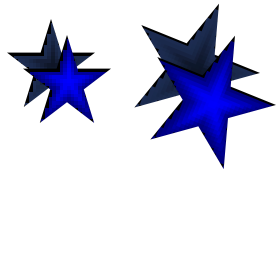
对 n 张牌进行 n 次洗牌，洗牌规则如下：在第 k 次洗牌时（ $k=1, \dots, n$ ），对第 i 张牌（ $i=1, \dots, n/k$ ）随机地产生一个小于 n 的正整数 d ，互换第 i 张牌和第 d 张牌的位置。



1.3 算法的数学基础

```
1. void shuffle(Type A[],int n)
2. {
3.     int i,k,m,d;
4.     random_seed(0);    /*初始化随机数的种子*/
5.     for (k=1;k<=n;k++) {
6.         m = n / k ;
7.         for (i=1;i<=m;i++) {
8.             d = random()%n; /*生成一个小于n的随机数*/
9.             swap(A[i],A[d]);
10.        }
11.    }
12. }
```

$$T(n) = \sum_{i=1}^n \frac{n}{i} = n \sum_{i=1}^n \frac{1}{i}$$



1.3 算法的数学基础

- 举例4：二分检索算法（假设 $n=2^k-1$ ）

算法 **BinarySearch** (T, l, r, x)

输入：数组 T ，下标从 l 到 r ；数 x

输出： j

1. $l \leftarrow 1; r \leftarrow n$

2. **while** $l \leq r$ **do**

3. $m \leftarrow \lfloor (l+r)/2 \rfloor$

4. **if** $T[m]=x$ **then return** m // x 中位元素

5. **else if** $T[m] > x$ **then** $r \leftarrow m-1$

- 实例个数： $2n+1$

- 查找概率： $p_i=1/(2n+1)$

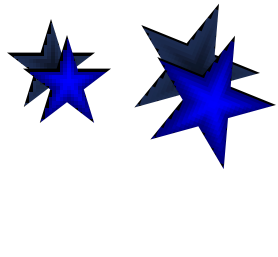
- 查找成功比较次数

$$\sum_{i=1}^k i2^{i-1}$$

- 查找不成功比较次数：

$$k(n+1)$$

$$T(n) = \frac{1}{2n+1} \left(\sum_{i=1}^k i2^{i-1} + k(n+1) \right)$$



1.3 算法的数学基础

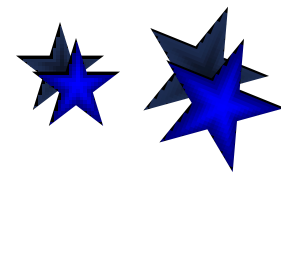
- 求和的方法

- (1) 等差、等比数列和调和级数求和

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}$$

$$\sum_{k=0}^n aq^k = \frac{a(1-q^{n+1})}{1-q}, \quad \sum_{k=0}^{\infty} aq^k = \frac{a}{1-q} \quad (q < 1)$$

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1) \quad \text{调和级数}$$



1.3 算法的数学基础

(2) 某些求和公式，求不出精确的值，可以估计和的上界

- 具体方法：使用放大法，使数列中的某些项放大，使数列变成一个类似于等比或等差数列的形式，然后再求和。
- 放大方法1：用序列中的最大项替代每一项

$$T(n) = \sum_{k=1}^n a_k \leq na_{\max}$$

优点：简单

缺点：放大后求得的可能与原数列的和差距过大。但如果放大后所求得的和函数的阶与原公式的阶保持不变，这种放大还是有意义的。



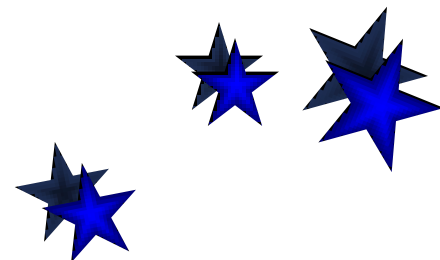
1.3 算法的数学基础

- 方大方法2：等比级数

假设存在常数 $r < 1$ ，使得 对一切 $k \geq 0$ 有 $a_{k+1}/a_k \leq r$ 成立。

$$T(n) = \sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$$

$$a_1 \leq a_0 r, a_2 \leq a_1 r \leq a_0 r^2, \dots$$



1.3 算法的数学基础

例7 估计 $\sum_{k=1}^n \frac{k}{3^k}$ 的上界.

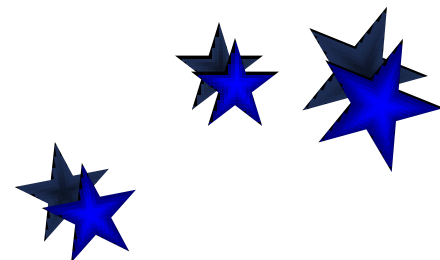
解:

$$a_k = \frac{k}{3^k}, \quad a_{k+1} = \frac{k+1}{3^{k+1}}$$

得

$$r = \frac{a_{k+1}}{a_k} = \frac{1}{3} \frac{k+1}{k} \leq \frac{2}{3}$$

$$\sum_{k=1}^n \frac{k}{3^k} \leq \sum_{k=1}^{\infty} \frac{1}{3} \left(\frac{2}{3}\right)^{k-1} = \frac{1}{3} \frac{1}{1 - \frac{2}{3}} = 1$$

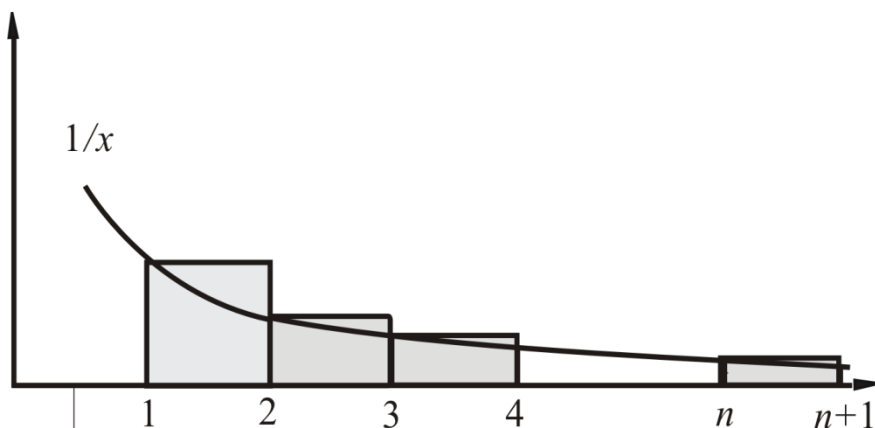


1.3 算法的数学基础

- 方大方法3：利用积分求出累加和的渐进的界——调合级数

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\ &= \ln(n+1)\end{aligned}$$



$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &= 1 + \sum_{k=2}^n \frac{1}{k} \\ &\leq 1 + \int_1^n \frac{dx}{x} \\ &= \ln n + 1\end{aligned}$$

